The Mathematics of QAOA

Quantum Alternating Operator Ansatz — Derivation and Routing Application

1 From Classical Cost to a Quantum Operator

QAOA solves problems of the form: find a bitstring $z \in \{0,1\}^n$ minimizing a classical cost function $C(z)$. Any QUBO problem

$$C(z) = \sum_{i,j} Q_{ij} z_i z_j$$

converts to an Ising Hamiltonian via $z_i \to \frac{1-s_i}{2}$, where $s_i \in \{-1,+1\}$, giving

$$H_C = \sum_{i<j} J_{ij}\, \sigma_i^z \sigma_j^z + \sum_i h_i\, \sigma_i^z.$$

This H_C is *diagonal* in the computational basis, so it simply returns the classical cost as an eigenvalue:

$$H_C\, |z\rangle = C(z)\ |z\rangle\,.$$

No quantum behaviour has been introduced yet — H_C is a faithful repackaging of $C(z)$ as an operator so a quantum computer has something to act on.

Worked case: weighted MaxCut

For a graph $G = (V, E)$ with edge weights w_{ij}, the cut cost is $C(z) = \sum_{(i,j)\in E} z_i \oplus z_j$. Using spins $s_i = 1 - 2z_i$, one has $s_i s_j = -1$ exactly when the edge is cut, so

$$C(z) = \sum_{(i,j)\in E} \frac{1 - s_i s_j}{2} \quad \Longrightarrow \quad H_C = \sum_{(i,j)\in E} \frac{w_{ij}}{2}\left(1 - \sigma_i^z \sigma_j^z\right).$$

2 The Mixer Hamiltonian

H_C alone cannot move amplitude between different bitstrings — every $|z\rangle$ is already an eigenstate. A second Hamiltonian that does *not* commute with H_C is required to drive exploration. The standard choice is the transverse-field mixer

$$H_M = \sum_i \sigma_i^x.$$

H_M generates independent bit flips: its eigenstates are superpositions of computational basis states, so evolving under it mixes $|z\rangle$'s together. Conceptually, H_C shapes the cost landscape (imprints phase), while H_M drives exploration (converts phase into amplitude change).

3 The QAOA Ansatz

Start from the uniform superposition

$$|s\rangle = |+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_z |z\rangle \,,$$

which is the ground state of $-H_M$ and is trivially prepared with one layer of Hadamard gates. Apply p alternating layers:

$$|\gamma, \beta\rangle = \underbrace{e^{-i\beta_p H_M} e^{-i\gamma_p H_C} \cdots e^{-i\beta_1 H_M} e^{-i\gamma_1 H_C}}_{p \text{ layers}} |s\rangle \,,$$

with $2p$ real variational parameters $\gamma = (\gamma_1, \ldots, \gamma_p)$, $\beta = (\beta_1, \ldots, \beta_p)$.

Why a phase step alone changes nothing measurable

Writing $|\psi\rangle = \sum_z c_z |z\rangle$, applying $U_C(\gamma) = e^{-i\gamma H_C}$ gives $c_z \to e^{-i\gamma C(z)} c_z$. Since $|e^{-i\gamma C(z)} c_z|^2 = |c_z|^2$, measurement probabilities are unchanged — U_C only imprints relative phase between bitstring amplitudes.

How the mixer converts phase into probability

For a single qubit, $U_M(\beta) = e^{-i\beta\sigma^x} = \cos\beta\, I - i \sin\beta\, \sigma^x$, so

$$|0\rangle \to \cos\beta\, |0\rangle - i \sin\beta\, |1\rangle \,, \qquad |1\rangle \to \cos\beta\, |1\rangle - i \sin\beta\, |0\rangle \,.$$

After U_M, the new amplitude of $|0\rangle$ is a *sum* of contributions from the old c_0 and c_1: $c_0' = \cos\beta\, c_0 - i \sin\beta\, c_1$. This is interference: the phase difference introduced by U_C now determines whether c_0 and c_1 combine constructively or destructively, changing actual measurement probabilities.

4 Gate Compilation

Because $H_M = \sum_i \sigma_i^x$ is a sum of commuting single-qubit terms,

$$U_M(\beta) = \prod_i e^{-i\beta\sigma_i^x} = \prod_i R_x(2\beta) \quad \text{(one } R_x \text{ gate per qubit)}.$$

For $U_C(\gamma)$, dropping the irrelevant global phase from the constant offset,

$$U_C(\gamma) = \prod_{(i,j)\in E} e^{i\frac{\gamma w_{ij}}{2} \sigma_i^z \sigma_j^z},$$

and each two-qubit term is compiled using the standard identity

$$e^{-i\theta\, \sigma_i^z \sigma_j^z} = \mathrm{CNOT}_{i,j} \cdot R_z^{(j)}(2\theta) \cdot \mathrm{CNOT}_{i,j}.$$

The first CNOT computes the parity of qubits i, j onto qubit j; R_z applies a parity-dependent phase; the second CNOT restores qubit j's computational value. Per edge: CNOT, R_z, CNOT — $3|E|$ gates per U_C layer.

5 The Classical Optimization Loop

The quantity being minimized is the expected cost

$$F_p(\gamma, \beta) = \langle \gamma, \beta | \, H_C \, | \gamma, \beta \rangle = \sum_z |c_z(\gamma, \beta)|^2 \, C(z).$$

This is estimated by sampling: run the circuit, measure, repeat N times, average $C(z)$ over the sampled bitstrings. A classical optimizer (COBYLA, SPSA, gradient descent) then updates (γ, β) to minimize $\hat{F}_p$:

$$(\gamma^*, \beta^*) = \arg\min_{\gamma,\beta} F_p(\gamma, \beta).$$

The minimization happens entirely in this classical loop — the quantum circuit is only a function evaluator at a given (γ, β); no single circuit run "solves" the problem. Gradients, when needed, use the parameter-shift rule:

$$\frac{\partial F_p}{\partial \gamma_k} = \frac{1}{2} \left[F_p\big(\gamma_k + \tfrac{\pi}{2}\big) - F_p\big(\gamma_k - \tfrac{\pi}{2}\big) \right].$$

After convergence, the circuit is sampled many times at (γ^*, β^*), and the lowest-cost bitstring actually observed is taken as the answer.

6 Why It Works: The Adiabatic Theorem

Consider the time-dependent Hamiltonian

$$H(t/T) = \left(1 - \frac{t}{T}\right) H_M + \frac{t}{T} H_C, \qquad t \in [0, T].$$

The adiabatic theorem states that if the system starts in the ground state of H_M and T is large enough, evolution stays in the instantaneous ground state throughout, ending in the ground state of H_C — which, by construction, is precisely the optimal bitstring z^*.

QAOA is a first-order Trotterization of this continuous path, discretized into p alternating steps. Formally,

$$\lim_{p \to \infty} \min_{\gamma,\beta} F_p(\gamma, \beta) = \min_z C(z),$$

with the optimal schedule converging to a discretized adiabatic ramp (Farhi, Goldstone, Gutmann, 2014). In practice p is small (1–10) due to circuit depth and noise limits, so QAOA is used as a heuristic rather than with the asymptotic guarantee.

7 Application: Routing on a Weighted Graph

Consider $G = (V, E)$ with node weights w_i (e.g. per-stage processing cost) and edge weights c_{ij} (e.g. transition cost). The goal is a valid path from source s to target t minimizing total cost.

Decision variables

$$x_{ij} \in \{0, 1\} \text{ (edge } (i, j) \text{ used)}, \qquad y_i \in \{0, 1\} \text{ (node } i \text{ visited)}.$$

Objective

$$C(x,y) = \sum_{(i,j)\in E} c_{ij}\, x_{ij} + \sum_{i\in V} w_i\, y_i.$$

Constraints as penalties

Flow conservation (degree-2 at interior nodes, degree-1 at s,t):

$$P_{\text{flow}} = \sum_{i\neq s,t} \Big(\sum_{j:(i,j)\in E} x_{ij} - 2y_i \Big)^2 + \Big(\sum_{j:(s,j)\in E} x_{sj} - 1 \Big)^2 + \Big(\sum_{j:(t,j)\in E} x_{tj} - 1 \Big)^2.$$

Consistency between edge and node variables:

$$P_{\text{link}} = \sum_i \Big(\deg(i)\, y_i - \sum_{j:(i,j)\in E} x_{ij} \Big)^2.$$

Full QUBO and Hamiltonian

$$H_C(x,y) = \sum_{(i,j)} c_{ij} x_{ij} + \sum_i w_i y_i + \lambda_1 P_{\text{flow}} + \lambda_2 P_{\text{link}},$$

with penalty weights λ_1, λ_2 chosen large enough that any constraint violation costs more than any objective saving. Expanding the squares yields a sum of linear and quadratic terms in x_{ij}, y_i, which converts to Ising form exactly as before, with one qubit per binary variable ($|E|+|V|$ qubits total).

Constraint-preserving alternative: the XY-mixer

Rather than penalizing invalid paths, an XY-mixer ($\sigma^x\sigma^x + \sigma^y\sigma^y$ terms) restricts mixing to bitstrings of fixed Hamming weight. If "valid path" corresponds to a fixed number of selected edges, the search never leaves the feasible subspace, removing the need for penalty terms at the cost of a more complex mixer circuit.